

1. PROJECT OVERVIEW

- **Project Title:** ComicCraft - AI Comic Story Creator using Gemini Models

2. INTRODUCTION The Coding and Solution document outlines the practical implementation of the core AI pipeline and documents the technical troubleshooting required to overcome real-world development hurdles, including network blocks and system limitations.

3. CORE IMPLEMENTATION LOGIC

A. API Routing and Pipeline Execution (`main_routes.py`) The application relies on asynchronous FastAPI routing to sequentially trigger the AI modules:

Python

```
@router.post("/create-comic", response_class=HTMLResponse)
async def process_comic_request(request: Request,
story_prompt: str = Form(...), character: str = Form(...)):
    # 1. Structure the Narrative
    formatted_prompt = f"Create a comic about {story_prompt}
featuring {character}."
    story_blueprint = fetch_gemini_outline(formatted_prompt)
    full_dialogue = fetch_gemini_story(story_blueprint)

    # 2. Render Visuals Iteratively
    downloaded_images = []
    for index, panel_data in enumerate(story_blueprint):
        image_url =
generate_panel_image(panel_data["image_prompt"])
        downloaded_images.append(image_url)

    # 3. Assemble and Export
    comic_layout = construct_layout(downloaded_images,
full_dialogue, story_blueprint)
    exported_file = compile_pdf(comic_layout)
```

```
    return templates.TemplateResponse("preview.html",
{"request": request, "layout": comic_layout, "pdf":
exported_file})
```

B. Resilient Image Processing (`image_handler.py`) To prevent application failure during API timeouts (HTTP 503/402), a robust fallback system is coded:

Python

```
def generate_panel_image(prompt,
filepath="static/panels/temp.jpg"):
    # Primary Execution: Hugging Face API
    try:
        hf_token = os.getenv("HF_API_KEY")
        # HTTPS request execution via urllib...
        return filepath
    except Exception as e:
        print(f"API Failed, deploying fallback mechanism:
{e}")

        # Secondary Execution: Python PIL Text-to-Image
        Generation
        fallback_img = Image.new('RGB', (512, 512),
color="#FFCC00")
        drawer = ImageDraw.Draw(fallback_img)
        drawer.text((50, 50), "VISUAL
UNAVAILABLE\nMEANWHILE...", fill="black")
        fallback_img.save(filepath)
        return filepath
```

4. TROUBLESHOOTING & ERROR RESOLUTION LOG

- **Issue 1: OS File Path Limitations ([`WinError 206`])**
 - *Context:* Encountered during the `pip install` phase for extensive AI libraries.

- *Resolution:* Modified the Windows Registry (`LongPathsEnabled = 1`) to bypass the default 260-character directory path restriction.
- **Issue 2: Terminal Script Execution Blocked (UnauthorizedAccess)**
 - *Context:* The system restricted the initialization of the Python Virtual Environment (`env\Scripts\activate`).
 - *Resolution:* Overrode the local execution policies using the PowerShell command `Set-ExecutionPolicy RemoteSigned -Scope CurrentUser`.
- **Issue 3: Package Import Failures (ModuleNotFoundError)**
 - *Context:* FastAPI and Uvicorn failed to import during local server startup.
 - *Resolution:* Re-synchronized the virtual environment by running a clean `pip install -r requirements.txt` ensuring all dependencies were mapped to the active environment.
- **Issue 4: Network DNS Blockage ([Errno 11001] getaddrinfo failed)**
 - *Context:* ISP-level restrictions prevented direct API calls to Hugging Face models.
 - *Resolution:* Coded a secondary local Python-PIL fallback image generator (as shown in the code snippet above) to guarantee continuous PDF compilation despite severe network failures.